

Machine Vision Final Project Report

J.J. Moncus

2026-01-03

0) “Follow the Elbow”, and Why I Didn’t Pursue It

My first ideas involved a small amount of pose estimation. In the end, the first was implausible given the data we’re working with.

Algorithm

- 1) For every frame:
 - Use template images to identify the maximum-likelihood estimate (x,y) coordinates of the right shoulder, elbow, and wrist
 - Fit a line connecting the shoulder to the elbow, and fit another line connecting the elbow to the wrist.
 - Calculate the angle between the two lines.
 - Each frame is thus reduced to a one-dimensional scalar (i.e. the angle of the elbow), and we no longer have to hold onto the pixel information.
- 2) Count the number of instances the elbow angle exits and re-enters “threshold zones” near 0° (for “closed”) and 180° (for open).
 - Since arms naturally do not close all the way, the precise angle threshold for closed would have to be picked by inspecting the training data, perhaps somewhere near 10° to start. The same goes for the open position - some individual’s may be unable to straighten their arms all the way, so we might declare open to be anywhere above 170° .

Why I think this wouldn’t work

- 1) The shoulder, elbow, and wrist all change orientation as they move, whereas the template image used to track them would likely be fixed (probably based on what each joint typically looks like in the initial push-up top position). Thus, we might lose track of the joint coordinates as the person executes the pushup, which would make computing lines through those points very noisy.
- 2) Not all videos even feature the right arm. We might have to start with some first layer, identifying what size of the person is visible, then apply a different template for each side, which seemed needlessly complicated.
- 3) Some videos are shot such that the elbow angle isn’t obvious (like from the front, or from above). In these cases, fitting useful lines could be very difficult.

- 4) This method would be sensitive to camera movement. If the elbow angle in plane-space looks like it's changing just because the camera rotates around the person, the model might see pushups where there aren't any.

In short, this approach *might* make sense if all videos were shot from the side, and if the orientation of the joints don't change much as they move (i.e. if it's still easy to recognize that a shoulder is a shoulder, whether the arm is straight or bent). Given none of this is the case, I scrapped this plan pretty quickly.

Learning About the Data

To get a sense of our training data, I watched most of our training videos and inspected some metadata:

- All videos seem to be shot with 24 frames-per-second, for a total length of roughly 90 to 360 frames (i.e. roughly 3.5 to 10 seconds).
- Most videos are shot in wide-screen, meaning if we squish videos to a 224x224 square resolution for training, the subject will look squished. Hopefully the push-up movement would still be apparent.
- Videos are shot from all angles (from the side, from the front, from above), and sometimes with camera movement, which validates that just using pose estimation for the subject would have been tricky.
- Since pushups are relatively smooth movements, our model may be able to stride over frames (i.e. for each video, only look at every 5th or 10th frame) and still see the subject's movement. This way, we could save a lot of memory overhead.

1) Learn the Features from Scratch (With Underwhelming Results)

I was inspired to see whether the task was possible without worrying too much about feature engineering. The promise of a deep learning approach is that the model may be able to learn useful data representations on its own, and I was excited to see what could be done by using raw videos.

In hindsight: all of these approaches suffered from severe problems:

- Underwhelming final training accuracy
- very long training time (30 epochs could easily take 3 hours).
- many epochs without improvement
- memory overload: training would consistently take up anywhere from 10GB to 14GB of unified MPS memory (on Mac)

In the end, I was nudged back to trying pose estimation, as a way of working with significantly smaller data representations that were more pertinent to our task.

But first, below is a record of what I tried:

1.1): A Basic CNN for both space and time

To track the subject's pushup movement over space and time, I proposed a two-part architecture

- A series of 2D-convolutions on each frame, to extract spatial features about the subject and its surroundings.
- A series of 1D-convolutions across frames, to see how those spatial features changed over time.

Stage	Input Shape	Operations	Output Shape
Spatial Encoder	$(B \times T, 3, 224, 224)$	6x "Conv2d (Stride 2) + ReLU" + Linear(640, 2)	$(B \times T, 2)$
...	$(B \times T, 2)$	expand and transpose	$(B, 2, T)$
Temporal Conv	$(B, 2, T)$	3x Conv1d (Stride 2) + ReLU	$(B, 18, 13)$
...	$(B, 18, 13)$	Flatten	$(B, 234)$
Classifier	$(B, 234)$	Linear	$(B, 1)$

Table 1: VideoConvModel: 27,659 parameters

Below is the summary of the first attempt, which I then modified a number of ways in different combinations:

Loss: Mean Squared Error, for regression

The open question was whether the spatial, 2D-Conv layers would learn enough about what a person is, and what it means for that person to move, with no pre-training.

i Disclaimer

I reached for a CNN before even trying a fully-connected MLP.

An MLP has much weaker inductive bias than a CNN, since the MLP does not presume translational equivariance over the input data. This is helpful sometimes in that an MLP could potentially learn far more complicated relationships between inputs and outputs, but at the cost of much larger memory overhead. However, the CNN would have the advantage of learning to understand pushups regardless of where the subject lives in the frame, whereas an MLP would have to relearn this whenever the subject appears in a new locatin. The weak inductive bias also makes the MLP more likely to overfit. For this reason, along with our memory footprint limits, a CNN seemed preferable.

In hindsight, starting with an MLP would have been helpful to get a baseline understanding for how difficult the our learning problem is. If even a relatively-deep MLP struggles to acheive high training accuracy, then our CNN would likely be insufficient. Luckily, this turned out to not be necessary.

The Result: Vanishing Gradients

Every attempt to use a CNN for building spatial features resulted in the model only ever predicting the mean label for each batch.

I tried modifying the CNN a number of ways:

- added Batch Normalization to each convolutional block in both spatial and temporal stages.
- several tweaks to the Linear bottlenecks in each stage to make the compression less aggressive (and adjusting the convolutions' input/output expectations accordingly).
- Increased temporal convolutions kernel size from the default 3 to 5, to try to capture more movement in each step.

Each tweaked model varied significantly in size, up to 669,569 parameters total.

Across 5 attempts with different adjustments, the model did not effectively train and only ever predicted the mean of the test batch.

1.2): ResNet for Spatial Features

After several failed attempts to solve the vanishing gradients problem, I thought a small ResNet with skip connections might do the trick. The skip connections make the loss landscape smoother, which makes calculating gradients easier.

So I replaced the spatial feature stage with pre-built ResNet18, beginning with all weights frozen to avoid the computational overhead of computing so many more gradients.

Loss: Mean Squared Error, for regression

Stage	Input Shape	Operations	Output Shape
Spatial Features	$(B \times T, 3, 224, 224)$	Pre-trained ResNet18 (Frozen)	$(B \times T, 512, 1, 1)$
...	$(B \times T, 512, 1, 1)$	Tranpose	$(B, 512, T)$
Temporal Processor	$(B, 512, T)$	3x [Conv1d + BatchNorm + LeakyReLU]	$(B, 32, 13)$
...	$(B, 32, 13)$	Flatten	$(B, 416)$
Classifier	$(B, 416)$	Linear	$(B, 1)$

Table 2: VideoResNetModel: 11,408,682 parameters

This architecture successfully trained, but only achieved about 52% training accuracy, with significantly higher training time.

Modification

ResNet18 is pre-trained on ImageNet, and thus understands a lot about various objects, including people and landscapes. However, it may not understand anything specific about pushups. So, I unfroze the final layer of the resnet backbone, to see if weight-updates would help the ResNet better recognize spatial features important for counting pushups.

This ultimately made the model worse (43% training accuracy, down from 52% earlier), so I refroze all ResNet-backbone weights going forward.

1.2.1) Switching to Classification

The first breakthrough came from switching to ten-class classification, using cross-entropy-loss, instead of regression. The original ResNet-backed hybrid architecture (2) achieved 61% training loss after the switch (after ~20 epochs).

In previous attempts, the model would be penalized for even small deviations from integer predictions, and in the end, the rounded, predicted number of pushups was often one off from the actual value, suggesting the model was getting close but not within rounding distance.

In hindsight, this should be intuitive - asking the model to pick amongst 10 buckets is likely easier than asking it to hit an exact spot on a continuous line.

All model attempts going forward used cross-entropy loss, instead.

1.2.2) Trying Data Transformations

I was concerned that the model might be severely inhibited by how small our dataset is. If I could implement similar transformations to those in Practical 9, then our model would see slightly different versions of training points during each epoch, which would artificially “expand” the size of our dataset and hopefully nudge the model to learn real push-up movement patterns instead of memorizing coincidental video details.

Online research suggested it was very important that, within a given video, transformations needed to be applied the same way to all frames. Random cropping, angle shifts, or color jitter applied non-uniformly would make the video look chaotic and warbly, making the learning problem much more difficult. So, I built wrapper

functions which generated a random seed per video and guaranteed the same random operations were copied across all frames in the same video.

I tried two different sets of training and validation transformations: one “difficult” and one “easier” (written below).

Difficult

```
## Copied from Practical 9
train_transform = T.Compose([

    T.Resize( (224, 224) ),
    T.RandomCrop(150),

    T.RandomHorizontalFlip(p = .1),
    T.RandomVerticalFlip(p = .1),
    T.RandomRotation(degrees = 10),

    T.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1),

    T.Normalize(mean = [ 0.485, 0.456, 0.406 ],std = [ 0.229, 0.224, 0.225 ])
])

val_transform = T.Compose([

    T.Resize( (224, 224) ),
    T.CenterCrop(150),
    T.Normalize(mean = [ 0.485, 0.456, 0.406 ],std = [ 0.229, 0.224, 0.225 ])
])
```

Easier

```
train_transform = T.Compose([

    # random crop is confined to the edges
    T.Resize((240, 240)),
    T.RandomCrop(224),

    # Removed vertical flip, so all pushups are still going downwards
    T.RandomHorizontalFlip(p=0.5),

    # Reduced from 10 to 5 degrees
    T.RandomRotation(degrees=5),

    # smaller brightness and contrast jitter, no saturation change
    T.ColorJitter(brightness=0.05, contrast=0.05),

    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

val_transform = T.Compose([

    # removed random crop
    T.Resize((224, 224)),
```

```
T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

In the end, both implementations made the model significantly worse, and the model required significantly more time to train up to these values (~60 epochs):

- **Difficult:** 36% final training accuracy
- **Easier:** 55% final training accuracy

The model was not learning enough about push-ups for added “push-up variation” to be helpful. It was just making the classification problem harder.

(Later, different transformations became very useful - more on that in section #.##).

1.2.3) Trying Transformer Architecture for Temporal Stage

Maybe the problem was that convolutions weren’t strong enough to track the spatial features’ movement over time. Given a video is naturally a sequence of frames, I imagined a small transformer for our temporal stage might improve things. The model could learn to attend to certain important frames in the sequence (like, perhaps, the beginning, when the subject is in the start position, to use as reference for determining when a pushup was completed).

Stage	Input Shape	Operations	Output Shape
Spatial Features	$(B \times T, 3, 224, 224)$	Pre-trained ResNet18 (Frozen)	$(B \times T, 512, 1, 1)$
...	$(B \times T, 512, 1, 1)$	Unpack dimensions	$(B, T, 512)$
Sequence Prep	$(B, T, 512)$	Concat CLS Token + Pos. Encoding	$(B, T + 1, 512)$
Transformer Encoder	$(B, T + 1, 512)$	2 Layers [4 Attn. Heads + Feed Forward]	$(B, T + 1, 512)$
...	$(B, T + 1, 512)$	extract CLS value in the front	$(B, 512)$
Classifier	$(B, 512)$	Linear	$(B, 10)$

Table 3: VideoTransformerModel: 15,439,434 parameters

The transformer did not help at all - despite many more parameters this architecture ultimately reached 25% training accuracy after 20 epochs.

I concluded this this architecture would likely need significantly more data to learn anything valuable.

1.2.4) The same temporal convolutions, but deeper.

The next unlock came from simply adding another layer each to the temporal and classification stages of the original resnet architecture:

After 25 epochs, this architecture was able to acheive 100% during some training epochs, with a final validation accuracy of 75%. It was clearly overfitting, but it was reassuring to see that an architecture like this could at least learn something.

Stage	Input Shape	Operations	Output Shape
Spatial Features	$(B \times T, 3, H, W)$	Pre-trained ResNet18 (Frozen)	$(B \times T, 512, 1, 1)$
...	$(B \times T, 512, 1, 1)$	Unpack dimensions	$(B, T, 512)$
Temporal Processor	$(B, 512, T)$	4x [Conv1d + BatchNorm + LeakyReLU]	$(B, 32, 7)$
...	$(B, 32, 7)$	Flatten	$(B, 32 \times 7)$
Classifier	$(B, 32 \times 7)$	2x [Linear + LeakyReLU]	$(B, 10)$

Table 4: VideoResNetModel: 11,726,522 parameters

1.2.5) Data Augmentation for Class Balance

It was at this phase I recalled the importance of class balance - it had not dawned on me until now that nearly all training videos contained 2, 3, and 4 pushups. Given we don't know the class balance of our test set, I needed the model to reliably predict anywhere from 1 to 10, so I needed a way of expanding the training set to learn higher pushup counts.

After some online research, I landed on the following augmentations:

Operation	Labels Gained
Cut all 2's in half	1's
Cut all 4's in half	2's
Extend 3's by about 2/3	5's
Double 3's	6's
Extend 4's by about 3/4	7's
Double 4's	8's
Triple 3's	9's
Double 5's (including previously-generated)	10's

Table 5: Data Stretching and Slicing Operations

In the end, we achieved a more balanced dataset, with the following label counts:

Label	1	2	3	4	5	6	7	8	9	10
Count	33	58	34	21	36	36	22	21	34	34

Table 6: Final training set labels and counts (329 total)

Slicing needed to be done with care to avoid jump cuts, which could be misinterpreted by the model as pushups. To keep transitions smooth, I concatenated "palindrome stitches", i.e. the *reverse* list of frames with the original, like so:

- Doubles = Original + Palindrome
- Triples = Original + Palindrome + Original
- 3 to 5 = Original + (Palindrome * 66%)
- 4 to 7 = Original + (Palindrome * 80%)

I used 66% and 80% instead of 60% and 75% stitches for the 5's and 7's because many videos contained a little bit of “dead space” at the beginning before the subject would first move, which would downstream lead the 5th and 7th push-ups to get cut off too early. The extra stitching helped ensure the final pushup was completed.

All cut-in-half videos were cut at the nearest integer to the midpoint frame.

The result was still bad: the ResNet Hybrid from (1.2.4) achieved just 19% training accuracy after 29 epochs

i Back to Regression after Data Augmentation

With a more balanced training set, I wondered whether regression would be more appropriate again. Perhaps earlier attempts were hamstrung by our training set's low variance. The answer is emphatically, “no:”

- After 30 epochs of training, ResNet Hybrid (1.2.4) achieved its best weights on epoch 3, despite a learning rate scheduler lowering the learning rate to $3e-7$ by epoch 25.
- This best model achieved just 6% accuracy.
- Some predictions were wildly high (two videos were predicted to contain 14 pushups, and another 16).

Lesson learned: regression is generally harder for this problem.

1.2.6) Simpler Augmentation, Just for Training set Size

As an experiment, I made a copy of every video by first flipping it horizontally and reversing the order of frames. This doubled the size of the training set, but retained the harsh class imbalance.

Unsurprisingly, ResNet Hybrid (1.2.4) achieved 75% accuracy on this set, this time after just 9 epochs. This mirrors the result from (1.2.4), and we can chalk up the faster convergence to the dataset being larger.

i Training Hygiene

Over the course of the failures throughout (1), I stumbled upon a number of helpful training tips to make life easier:

- **weight decay:** After each batch, the typical weight update is dictated by the learning rate. But to keep the model from overfitting, we can multiplicatively pull the model towards smaller updates using a small weight-decay. I tinkered with values of $1e-4$ to $9e-4$, with varying degrees of success.
- **A learning rate scheduler:** If the learning rate is too high, the weight updates may be too large, which can keep us from descending towards a minimum in the loss landscape. After a few epochs of no improvements, you can “schedule” your learning rate to drop by some percentage, making it more likely that an update will lead you downhill. I tinkered with schedule drops of $1/2$ to $1/10$ th the previous learning rate after 2 or 3 epochs of no improvement in validation loss.
- **AdamW:** The AdamW optimizer is specifically designed to decouple learning rates and weight decay (REVIEW THIS).
- **Early stopping:** I eventually implemented a rule where the training loop would end early if the learning rate ever fell below $1e-8$. At that rate, weight updates are unlikely to be meaningful.
- **Saving your Best:** You don't necessarily have to keep the model returned after the final epoch. Sometimes, the weights associated with lowest validation loss (and thus the one which in theory will generalize best) will happen sometime in the middle of training. I eventually implemented a section of my training protocol which, anytime a new “best validation loss” was achieved would save a “checkpoint” of the model state, optimizer state, scheduler state, and epoch when the loss occurred. That way, I always had access to the best model “so far.”

I implemented each of these in my final, successful attempts (2).

2) Use Post Estimation to Follow the Whole Person

While it is true that the subject's relationship with its surroundings (like the ground) changes over the course of a push-up, the essence of a push-up is mostly about the body's joints' orientation to each other:

- the shoulders and wrists coming together as the elbow moves backwards,
- the torso and back lowering and raising
- the feet staying fixed

Tracking these movements alone may be enough for our task. This approach also allows us to use significantly lower-dimensional data for every frame, which speeds up training.

2.1) The Same Data Augmentation from (1.2.5)

Our efforts with the ResNet were not all a waste! We can achieve a class-balanced training set by applying the same transformations as in (1.2.5), this time on pose estimations.

As before, we end up with 329 total training “videos” (pose tensors) distributed roughly evenly over labels 1 through 10.

First Attempt

I tried returning to our roots: a CNN + Linear Classifier, this time after pre-processing videos through MediaPipe. This model could quickly train 100 epochs on our 329 preprocessed pose files in about a minute.

Stage	Input Shape	Operations	Output Shape
Preprocess Videos	(B, C, T, H, W)	Mediapipe Joint Coordinate Estimation	$(B, T, 33, 4)$
...	$(B, T, 33, 4)$	Flatten and transpose	$(B, 132, T)$
Temporal Processor	$(B, 132, T)$	2x [Conv1d + BatchNorm + LeakyReLU]	$(B, 256, 1)$
Classifier	$(B, 256, 1)$	Flatten + Linear + LeakyReLU + Linear	$(B, 10)$

Table 7: PoseModel: 179,482 parameters

Training accuracy: 72% on our expanded class-balanced training set, 61% on the original training set

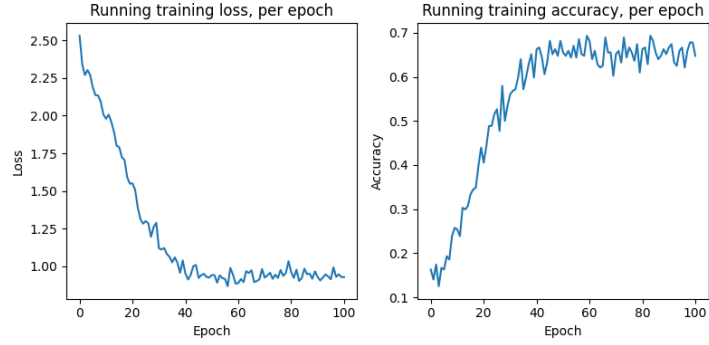


Figure 1: Training loss and accuracy, per epoch

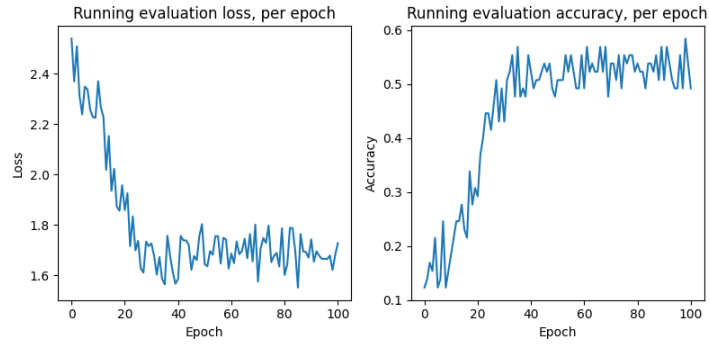


Figure 2: Validation loss and accuracy, per epoch

Our confusion matrix shows some peculiar behavior.

- Most mistakes are at the margin, predicting one up or down of the actual value (which is good).
- The model has a tendency to predict 6 on high-count videos. 6 is relatively highly represented in the dataset, but not by much.
- There are few instances of gross misses (e.g. confusing 8 for 2, or 2 for 7, etc.) These don't seem to be comprehensively explained by our data augmentation (if it were, we would expect more confusions between 3 and 5, or 4 and 7).

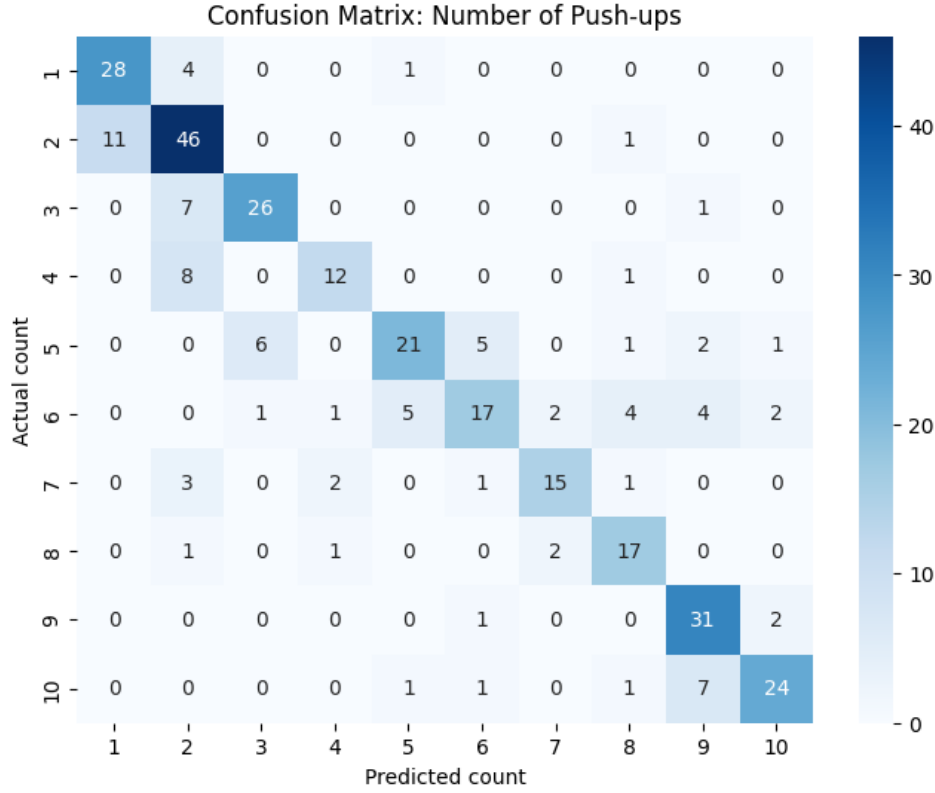


Figure 3: Confusion matrix, predicted vs. actual push-up counts

Small Modifications

Now that the time cost of training was so low, I could play with several tweaks:

- Adding small FC layer before temporal convolutions
 - it became helpful to allow the model to learn a more compressed representation of the joint coordinates before passing this representation to the temporal convolutions.
 - Expanded representations (i.e. going from 132 joint*features to 256) also helped, though less so than compression.
 - Adding an FC layer with the same in and out channels made the model worse.
 - **Takeaway:** Layers with the same input and output channels can be dangerous, the model is likely to learn non-useful representations, like rotation, translation, or simply noise.
- augmenting frame stride and max frame length (holding architecture constant)
 - stride 10, max frame length 100: 93.5% accuracy
 - stride 5, max frame length 200: 89% accuracy
 - stride 3, max frame length 300: 72% accuracy

- stride 10, max frame length 200: 7% accuracy
- **Takeaway:** Since all videos are going to be under 1000 frames, ensuring $stride \times maxlength = 1000$ guarantees we don't cut off videos too early. The longest video in our class-balanced set was FRAME COUNT. For videos at this speed, a stride of 10 seems to capture enough of the push-up movement to be meaningful. Lower strides run the risk of capturing glitchy movement, which might inflate the pushup count prediction. And if $stride \times maxlength < 1000$, then we end up padding the video with too much “stand-still” frames, which wash out the signal once our temporal layer hits the Global Average Pool at the end.
- weight decay
 - choosing the right weight decay called for balancing with the model architecture. A slightly larger model would sometimes overfit quickly, and the learning rate scheduler would sometimes exit the training loop after far fewer than 100 epochs. With these models, increasing weight decay had the dual advantage of tempering overfitting while allowing the model to train for longer before hitting the learning rate limit.

The Winning Model

Stage	Input Shape	Operations	Output Shape
Preprocess Videos	(B, C, T, H, W)	Mediapipe Joint Coordinate Estimation	$(B, T, 33, 4)$
...	$(B, T, 33, 4)$	Flatten	$(B, T, 33 \times 4)$
Spatial Embedding (FC)	$(B, T, 33 \times 4)$	Linear + LeakyReLU	$(B, T, 32)$
...	$(B, T, 32)$	transpose	$(B, 32, T)$
Temporal Processor	$(B, 32, T)$	3x [Conv1d + BatchNorm + LeakyReLU]	$(B, 256, 1)$
Classifier	$(B, 256, 1)$	Flatten + Linear + LeakyReLU + Linear	$(B, 10)$

Table 8: PoseModel: 164,538 parameters

85% accuracy on larger class-balanced training set, 93.5% accuracy on original training set

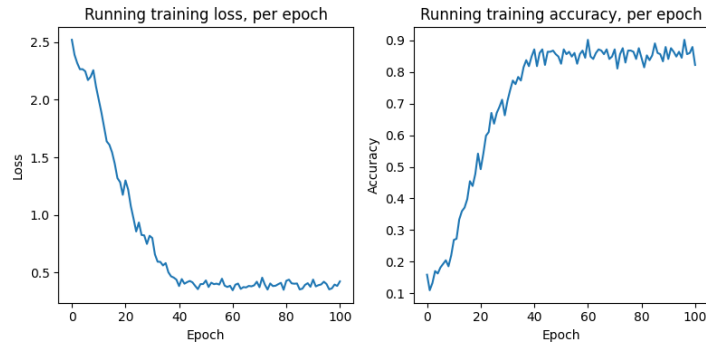


Figure 4: Training loss and accuracy, per epoch

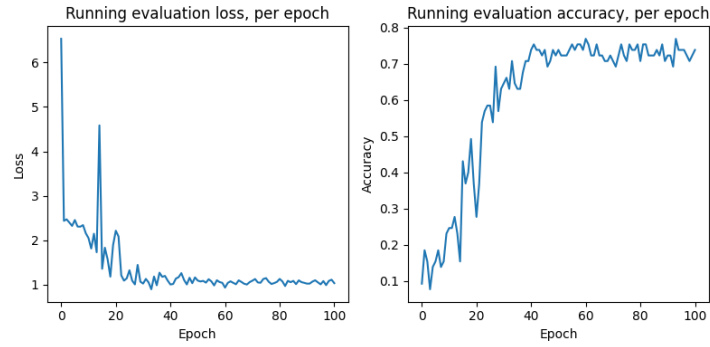


Figure 5: Validation loss and accuracy, per epoch

Our confusion matrix shows nearly all mistakes are on the margin, with the model predicting one up or down of the actual value.

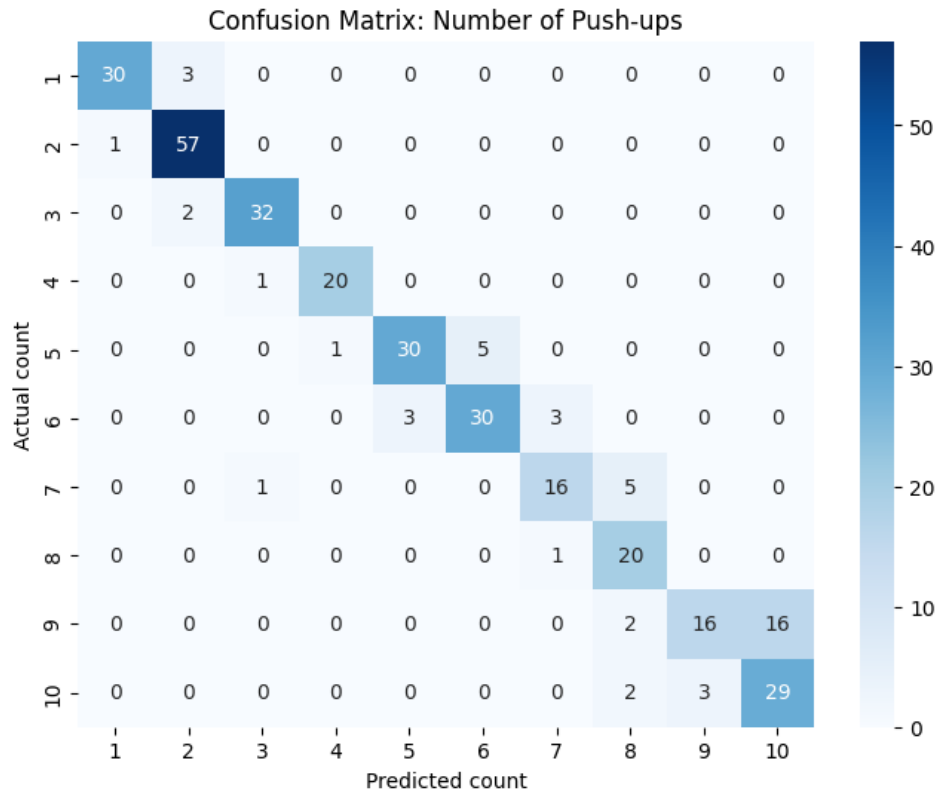


Figure 6: Confusion matrix, predicted vs. actual push-up counts